Feasibility-Adaptive Portfolio Seeding for Multi-Objective Spacecraft Payload Packing

Ronit and [Second Author]

***Abstract*—Stowing cargo in a spacecraft logistics module is a three-dimensional packing problem with two conflicting goals: keep the centre of mass (CoM) near its target and keep items reachable by the crew. Decoder-based multi-objective evolutionary algorithms fit this problem, yet on heterogeneous cargo they rarely return a valid packing. Plain NSGA-II succeeds in at most 4 of 30 seeds on five of eight synthetic instances, and never on Bischoff–Ratcliff instances with 20 or more box types. The two usual remedies each fail somewhere. A decreasing-volume Baldwinian repair restores feasibility but trades accessibility for compactness, and seeding one constructive solution helps only when that solution is feasible.**

We propose feasibility-adaptive portfolio seeding (FAPS), which seeds the population with best-fit-decreasing constructions under four orders and switches the repair on only if no valid packing has appeared after 10% of the generations. On 20 instances (eight synthetic with 50–500 items, twelve adapted from Bischoff–Ratcliff classes BR1–BR10) and about 3,100 runs at matched evaluation budgets, FAPS has the best mean rank (1.35 against 2.30–3.80; Friedman p = 4.5 $\times 10^{-8}$). It is significantly worse than a competitor on one instance, and on seeds where both methods are feasible it recovers the accessibility the repair gives up. The results hold under SMS-EMOA and under perturbed objective constants. A contact-constrained decoder lowers feasibility for every method. We recommend that packing studies report feasibility separately from front quality.

***Index Terms*—spacecraft logistics; three-dimensional packing; container loading; multi-objective evolutionary algorithms; constraint handling; population seeding; repair; hypervolume**

I. Introduction

Cargo for a crewed spacecraft must be packed so that the vehicle's centre of mass stays within tolerance. It must also stay retrievable by a crew who reach in through an access face and may have to remove items to reach the one they need. The two goals conflict. Compact packings that keep the CoM on target tend to bury items, and accessible packings tend to load mass unevenly. Multi-objective evolutionary algorithms (MOEAs) suit the problem because they return a set of compromises, from which an operator can pick.

A decoder-based MOEA searches over packing sequences and orientations, and a placement heuristic converts each chromosome into a layout. This design is standard in container loading [1]. Its weak point is that the decoder may fail to place every item, so a chromosome can describe an invalid packing. Constrained-domination selection [2], [3] steers the search toward feasibility but does not guarantee it. We find that for heterogeneous cargo it usually fails. An evaluation that compares fronts without separating feasibility then compares invalid layouts, or rewards whichever method finds any valid one.

Constructive heuristics such as best-fit-decreasing by volume handle feasibility well but return a single layout. Their order can be injected into the MOEA at decoding time, by repairing failed chromosomes [4], or at initialisation, by seeding the population [5]. We find that each remedy works in some regimes and fails in others. Repair restores feasibility everywhere but pulls the front toward CoM compactness and away from accessibility. Seeding gives better fronts when the seed is feasible and does little when it is not. On near-homogeneous cargo, either can do worse than no prior at all. A method that adapts to the instance is the obvious next step, and it is what we propose. The contributions are:

1. Feasibility-adaptive portfolio seeding (FAPS) (Section III-E), which seeds NSGA-II with constructive solutions under several orders and activates a volume-ordered repair only if the search finds no valid packing early on. On 20 instances, FAPS has the best mean rank and is significantly worse than a competitor on only one instance (Section VI-B).
2. An account of where the standard remedies fall short (Sections VI-A, VI-C and VI-D). On seeds where both methods are feasible, the repair leaves hypervolume unchanged and moves the front along the trade-off. Single seeding wins at scale and fails when its seed is infeasible. On a public benchmark, whether any prior helps depends on how heterogeneous the cargo is.
3. An evaluation protocol that reports feasibility separately from front quality (Section IV). It uses matched evaluation budgets, feasible-run rates, pooled feasible reference fronts, seed-paired tests with family-wise correction, and a comparison restricted to seeds on which both methods are feasible. It also includes an instance-level Friedman test, a second optimiser, an ablation, a sensitivity analysis and a contact-constrained decoder.
4. A corrected and tested batched GPU scorer (Sections V and VI-I). The original scorer treated padding rows as items, which biased the accessibility objective of every infeasible layout.

We do not validate the accessibility surrogate against crew data or model launch loads. Section VIII states these and other limits.

II. Related Work

Container loading. Bischoff and Ratcliff [6] set out the practical issues that distinguish container loading from abstract packing, and introduced the benchmark we adapt in

Ronit is with the Department of Computer Science and Engineering, Amity University, Noida, Uttar Pradesh, India (e-mail: ronitm0108@gmail.com).

[Second Author] is with [Department, Institution, City, Country] (e-mail: [email]).

Code, instances and derived data are available in the project repository.

Section VI-D. Bortfeldt and Wäscher [7] review the constraints that arise in practice, including weight distribution, stability, orientation, loading priorities and positioning. In their terms, our CoM objective is a weight-distribution criterion and our accessibility objective is a positioning criterion. Davies and Bischoff [8] give an early treatment of weight distribution and used the extended Bischoff–Ratcliff classes. Ramos et al. [9] enforce static mechanical equilibrium, a stricter stability model than the contact constraint we test. Ceschia and Schaerf [10] address multi-drop loading, where items must be reachable in delivery order, an accessibility requirement related to ours. Zhao et al. [11] compare container loading algorithms on common benchmarks.

Three-dimensional packing heuristics. Martello et al. [12] established exact and heuristic methods for three-dimensional bin packing. Crainic et al. [13] introduced Extreme Points (EP) as placement candidates, which our decoder uses. Parreño et al. [14] use maximal spaces instead. Gonçalves and Resende [1] encode a packing as a sequence and an orientation vector decoded by a placement heuristic, evolved with a biased random-key genetic algorithm. Our representation follows this decoder-based design.

Space cargo. Cargo accommodation for the International Space Station has been supported by an operational tool, CAST, which combines global optimisation with heuristics and balancing rules [15]. Fasano [16] treats non-standard packing problems from this domain. Ho [17] reviews space logistics optimisation at the campaign and network level. We did not find a study that pairs a CoM objective with an accessibility objective under an EP decoder and compares methods at a matched budget, though our search may have missed one.

Constraint handling, repair and seeding. NSGA-II [3] handles constraints by constrained domination, following Deb's [2] feasibility rules. Repair maps an infeasible solution to a feasible or less infeasible one, and Salcedo-Sanz [4] surveys repair methods as constraint-handling techniques. A repair whose phenotype is evaluated but not written back to the genotype is called Baldwinian; one that writes it back is Lamarckian [18]. Seeding inserts known good solutions into the initial population. Friedrich and Wagner [5] show that it can speed up MOEAs considerably, and also that it can bias the final population. FAPS combines a portfolio of seeds with a repair that is activated only on evidence of infeasibility. We use the pymoo implementations of NSGA-II and SMS-EMOA [19].

Evaluation methodology. Hypervolume [20] and the wider methodology of quality indicators [21] underpin our front comparisons. For comparing stochastic algorithms over several problems we follow Derrac et al. [22]: per-problem nonparametric tests with a family-wise correction, here Holm's [23] procedure, together with a Friedman test on mean ranks over problems. Cliff's [24] δ serves as the effect size.

GPU evaluation. Özcan et al. [25] run a grouping genetic algorithm for one-dimensional bin packing on a GPU. Our batched scorer addresses three-dimensional objectives and is reported only as a scoring-stage measurement.

III. Problem and Methods

A. Representation and decoder

A solution is a permutation σ of the N items together with an orientation vector o $\in \{1,\ldots,6\}^N$. Each orientation selects one of the six axis permutations of an item's width, depth and height. Where an item may use only some orientations (Section VI-D), the gene indexes the permitted ones cyclically. With all six permitted, the gene maps to itself.

The Extreme Point decoder places items in the order σ. Each item is tried, in its chosen orientation, at the current extreme points sorted by z, then y, then x. It is placed at the first point where it lies inside the container and overlaps no placed item, and the extreme points are then updated. An item with no valid point is left unplaced.

The access face is the plane z = 0, and z is depth from that face, so the decoder fills the container from the access face inward. By default an item need not touch the items nearer the access face. Section VI-G studies a contact-constrained variant.

B. Constraints and objectives

A layout is feasible when every item is placed. The constraints are g_1 = total unplaced volume ≤ 0 and g_2 = number of unplaced items ≤ 0.

f_1, CoM deviation (minimise). The Euclidean distance between the system centre of mass and a target point t. The system includes a dry mass of 5000 kg located at t. On the synthetic instances t = (0.5, 0.5, 1.0) m; Section VI-H also tests a target at the container centre. On the Bischoff–Ratcliff instances t is the container centre.

f_2, accessibility (maximise; implemented as minimising $-$CASC). For each packed item i, CASC_i = exp($-$c_i), with c_i = c_reach + c_occ + c_ext:

- Reach: c_reach = (z_i / L_arm)² with L_arm = 0.75 m.
- Occlusion: c_occ = $\lambda(1 - \text{v_i})$ with $\lambda = 1$, where v_i is the visible fraction of item i's footprint on a 10×10 grid, counting only items nearer the access face.
- Extraction: c_ext = Σ_j ρ_j (C_grasp + κ m_j), summed over the transitive blockers j of item i, with C_grasp = 2, κ = 0.05 kg^{-1}, and ρ_j = 0.4 for consumable or waste items and 1 otherwise.

The objective is the mean CASC over packed items. A layout with no packed item receives f_1 = 9999 and f_2 = 0.

C. Optimisers

NSGA-II. We use NSGA-II with constrained domination on (g_1, g_2). Population and generations are 100 and 100 for N $\leq$ 121, and 50 and 50 for N $\geq$ 250. Crossover applies OX1 order crossover to σ and uniform crossover (mask 0.5) to o, with probability 0.9. Mutation applies swap or inversion to σ and resets each orientation gene, each with probability 1/N. Duplicates are eliminated.

SMS-EMOA. Section VI-E uses SMS-EMOA with the same operators, population and generations. It differs from NSGA-II only in its survival rule: least hypervolume contribution among feasible solutions, and constraint violation among infeasible ones.

D. Constructive priors used as baselines

- Volume repair (Baldwinian). If decoding leaves an item unplaced, the chromosome is re-decoded with σ stably sorted by decreasing item volume and the same orientations. The decode with less unplaced volume is kept, and the genotype is not modified.
- Seeded NSGA-II. The initial population contains one BFD-V chromosome: the decreasing-volume sequence with the orientations that BFD-V chose. A test verifies that it decodes to exactly the BFD-V layout. The rest of the population is random.
- Large-item repair (Section VI-C only). Like the volume repair, but only items at or above the median volume move to the front. The smaller items keep their order from σ.

E. Feasibility-adaptive portfolio seeding (FAPS)

FAPS follows from two findings. The constructive order is what restores feasibility (Section VI-F), and forcing it on every failed evaluation distorts the trade-off (Section VI-C). A single seed helps when it is feasible and hardly at all otherwise. FAPS therefore starts from several constructive orders and keeps the repair as a fallback:

Seed encoding. Each seed is encoded so that its chromosome decodes to the constructive layout exactly.

Cost. The portfolio costs four constructions before the search. Their decoder calls are counted in the reported totals. The repair, once active, costs one extra decode per failed evaluation, as the volume repair does. FAPS has one parameter beyond NSGA-II, the activation point of 10% of the generations, which was fixed before any FAPS run and not tuned.

Activation in practice. Across the 440 FAPS runs, the repair never activated when at least one seed was feasible (SC1-100, SC1-250, SC1-500, SC2-250, SC2-500, BR1-2 and BR4-2). When no seed was feasible it activated at generation 10 in 228 of 230 runs.

F. Further baselines

We also run plain NSGA-II; random search with the same number of evaluations as NSGA-II on the same instance and seed; and three deterministic constructions that use the same EP decoder with all permitted rotations at each extreme point: First-Fit (FF) in dataset order, BFD-V by decreasing volume, and BFD-P by decreasing priority.

IV. Experimental Design

Synthetic instances. There are twelve synthetic instances (HyperDock-Bench V1) with N $\in$ {50, 100, 250, 500} in three scenarios: SC1 is consumable-heavy at 80% volume utilisation, SC2 is 40% safety items at 80% utilisation, and SC3 has SC1's item mix at 96% utilisation.

Item sizes follow cargo-transfer-bag classes plus an irregular class. Masses are lognormal by category, and each container is a cube sized to the target utilisation. Every instance is seeded from its identifier, and the generator reproduces all twelve files exactly (`docs/DATASETS.md`). No method finds a valid packing on any SC3 instance, so SC3 is reported in Section VI-A but excluded from the tests.

External instances. Twelve instances are adapted from the Bischoff–Ratcliff benchmark, classes BR1–BR10. Section VI-D describes them.

Runs. On the eight SC1 and SC2 instances, NSGA-II, the volume repair, seeded NSGA-II, FAPS, random search and the large-item repair each ran on seeds 1–30. On the Bischoff–Ratcliff instances every method ran on seeds 1–10. The SMS-EMOA study (Section VI-E), the ablation (Section VI-F) and the contact study (Section VI-G) use 10 seeds each. Including SC3 and the deterministic constructions, the study comprises about 3,100 runs. Every evaluated chromosome is archived, and comparisons use all evaluations, including those outside the final population. All objective values come from the CPU reference scorer.

Platform. A laptop with 28 logical CPU cores and an NVIDIA RTX 4050 Laptop GPU, running Python 3.12, pymoo 0.6.2, numpy 2.5.4, scipy 1.18.1 and numba 0.68.1.

Metrics. The feasible-run rate is the fraction of seeds with at least one feasible evaluation. For hypervolume (HV), the feasible points of all methods and seeds in a study are pooled per instance and min-max normalised on that pool, and HV is computed with reference point (1.1, 1.1). A run with no feasible point has HV 0. HV values can be compared within an instance and study, but not across studies with different method sets. IGD is the distance to the non-dominated pooled set and is defined for feasible runs only.

Statistics. Per instance, we use two-sided Wilcoxon signed-rank tests paired by seed, Cliff's δ as effect size, and bootstrap 95% intervals of the mean (10,000 resamples). Holm correction is applied across instances within each comparison. Comparisons where both methods have HV 0 in every run are excluded from the family, because no test is possible there. Across instances, we use a Friedman test on per-instance mean HV over all 20 instances, followed by pairwise Wilcoxon tests over instances.

With n seeds, the smallest attainable two-sided p-value is $2/2^n$: 0.002 for 10 seeds and 1.9×10^{-9} for 30. With Holm correction over 12 Bischoff–Ratcliff instances, tests on 10 seeds have limited power.

V. Implementation Correctness

Batched-scorer defect. A batched PyTorch scorer packs a population on the CPU and evaluates the accessibility objective on the GPU. It passed unplaced items as zero rows and scored them as real items at the origin, with errors up to about 0.1 in mean CASC. Fully packed layouts were unaffected, so an equivalence check on them passed, yet most layouts visited during search are partially packed. The corrected scorer masks padding rows. It matches the CPU reference to within 8×10^{-8} on partially packed layouts, and regression tests cover that case. An earlier analysis of this system used the defective scorer and is withdrawn (Appendix A).

Reproducibility checks. Pareto points from the laptop runs were re-decoded and re-scored in a separate Linux environment with different package versions, and the objective values

```
FAPS(instance, budget):
    for each order k in {volume, largest face area, longest side, mass}:
        s_k <- best-fit-decreasing construction under order k   # all permitted rotations
    P0 <- {chromosome of s_k for each distinct k} U random individuals
    repair_active <- false
    run NSGA-II (or SMS-EMOA) from P0 with the volume repair gated by repair_active:
        after each generation g >= ceil(0.1 * G):
            if no feasible packing has been evaluated so far: repair_active <- true
    return the archive of all evaluated solutions
```

Fig. 1. Feasibility-adaptive portfolio seeding (FAPS).

were identical. The deterministic heuristics produced identical layouts in both environments. The test suite (52 tests) checks that the generic implementation of every variant reproduces the published methods exactly, that every constructive seed decodes to its construction's layout, that FAPS's inactive repair leaves evaluations unchanged, and that every Bischoff–Ratcliff box respects its orientation flags.

VI. Results

A. Feasibility and hypervolume on the synthetic instances

Plain NSGA-II rarely returns a valid packing. It does so in at most 18 of 30 seeds on any instance, never on SC1-100 or SC2-250, and in at most 4 seeds on SC1-250, SC1-500 and SC2-500. Random search never does. On SC3 (96% utilisation) no method, including the three heuristics, packs every item.

The single-prior baselines split by instance. The volume repair is feasible in 87–100% of seeds. Seeding is significantly better than the repair on all four instances with 250 or more items and on SC2-50; the repair is better on SC1-50 and SC1-100.

FAPS against all three. FAPS beats plain NSGA-II on all eight instances. It is never significantly worse than seeding and is better on three (SC1-500, SC2-100, SC2-500). It beats the repair on five and is worse on one, SC1-100. Its feasible-seed counts match or exceed both baselines everywhere except SC1-50 (21, against 26 for the repair).

Why FAPS loses on SC1-100. There the volume seed is feasible, so FAPS behaves like seeded NSGA-II and inherits seeding's weakness on that instance. The repair produces the best fronts on SC1-100 under both optimisers (Section VI-E), and this is the only instance where FAPS loses consistently.

B. Overall ranking

The Nemenyi critical difference at $\alpha = 0.05$ for four methods and 20 instances is 1.05. By that conservative criterion, FAPS ranks significantly better than seeded and plain NSGA-II. Its margin over the volume repair, 0.95, is just below the critical difference, although the instance-level Wilcoxon test is significant even after a three-way Holm correction (p = 0.005).

C. What the priors do to the trade-off

Table I compares runs that differ in feasibility. Table III restricts each comparison to seeds on which both methods are feasible, and compares hypervolume and the best value each run reaches on each objective.

The repair moves the front along the trade-off. On every instance where enough seeds are feasible for both, the repair's hypervolume is indistinguishable from plain NSGA-II's, while its fronts reach significantly lower CoM deviation and significantly lower accessibility. We suspect the following mechanism: re-decoding from largest to smallest places the largest items first, at the access face, which keeps mass compact but buries small items behind large blockers.

Seeding improves the front at scale. On all four instances with 250 or more items, seeding beats the repair on both objectives in every comparison, each significant; the table shows two.

FAPS removes the repair's accessibility penalty. Against the repair on shared feasible seeds, FAPS reaches significantly higher accessibility on every synthetic instance except SC2-50, where there is no difference, and SC1-100, where it is lower. Its CoM deviation is equal or better except on SC1-50 and SC1-100. On the 500-item instances it also improves on single seeding in both objectives, which suggests the gain comes from having several starting structures.

The large-item repair partly fixes the penalty, at a cost. It moves only the large items to the front. On SC2-100 it recovers part of the repair's accessibility penalty (CASC 0.253 against 0.208, p = 1×10^{-4}), but it is feasible in fewer seeds (20 of 30), and it is significantly worse than the volume repair in overall HV on SC1-50, SC1-500 and SC2-500. FAPS recovers accessibility without losing feasibility or HV.

D. External benchmark: Bischoff–Ratcliff instances

We adapted the container-loading instances of Bischoff and Ratcliff [6], distributed through OR-Library. Classes BR1–BR7 come from `thpack1`–`thpack7`. Classes BR8–BR10 come from the extended set regenerated by S. Allen (OR-Library `br.zip`), as used by Davies and Bischoff [8]. We took problems 1 and 2 of six classes that span cargo heterogeneity: BR1 (3 box types), BR4 (10), BR7 (20), BR8 (30), BR9 (40) and BR10 (50).

The container is the benchmark's 587 × 233 × 220 cm box, with its door as the access face. The benchmark targets volume maximisation, and each problem holds about one container's volume of boxes. Our model requires every item to be placed, so each problem was adapted, before any run, as follows:

1. **Subset.** The boxes were expanded, shuffled with the problem's own seed, and taken up to 80% of container

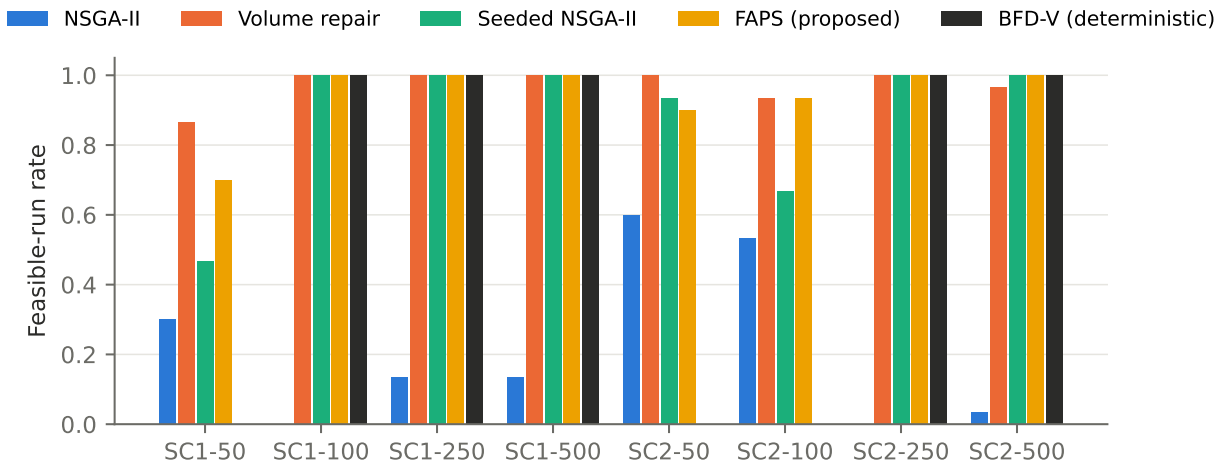


Fig. 2. Feasible-run rate per instance and method (30 seeds). Random search, FF and BFD-P are infeasible everywhere and are omitted.

TABLE I

SYNTHETIC INSTANCES, 30 SEEDS. EACH CELL GIVES THE MEAN HYPERVOLUME WITH ITS BOOTSTRAP 95% INTERVAL AND THE NUMBER OF FEASIBLE SEEDS OUT OF 30. THE REFERENCE FRONT POOLS ALL FIVE METHODS. THE BEST MEAN PER ROW IS IN BOLD. THE LAST THREE COLUMNS GIVE HOLM-ADJUSTED P-VALUES FOR FAPS AGAINST EACH COMPETITOR, MARKED + WHERE FAPS IS BETTER AND − WHERE IT IS WORSE.

Instance	NSGA-II	Volume repair	Seeded NSGA-II	FAPS	vs NSGA-II	vs repair	vs seeded
SC1-50	0.100 [0.042, 0.167] · 9	**0.371** [0.297, 0.439] · 26	0.211 [0.125, 0.303] · 14	0.338 [0.243, 0.435] · 21	+ 9.4×10^{-4}	1.0	0.34
SC1-100	0 · 0	**0.581** [0.539, 0.629] · 30	0.278 [0.223, 0.340] · 30	0.248 [0.201, 0.305] · 30	+ 4.5×10^{-6}	− **9.3** $\times 10^{-8}$	0.73
SC1-250	0.072 [0.014, 0.143] · 4	0.327 [0.308, 0.347] · 30	**0.679** [0.627, 0.735] · 30	0.609 [0.570, 0.649] · 30	+ 2.2×10^{-8}	+ 1.5×10^{-8}	0.25
SC1-500	0.066 [0.016, 0.132] · 4	0.171 [0.163, 0.179] · 30	0.388 [0.324, 0.449] · 30	**0.700** [0.626, 0.774] · 30	+ 2.8×10^{-8}	+ 1.5×10^{-8}	+ 2.1×10^{-7}
SC2-50	0.211 [0.139, 0.285] · 18	0.364 [0.321, 0.406] · 30	**0.476** [0.401, 0.548] · 28	0.361 [0.293, 0.427] · 27	+ 0.013	1.0	0.29
SC2-100	0.270 [0.174, 0.367] · 16	0.391 [0.330, 0.449] · 28	0.322 [0.230, 0.413] · 20	**0.524** [0.448, 0.594] · 28	+ 4.7×10^{-4}	+ 0.030	+ 0.021
SC2-250	0 · 0	0.315 [0.288, 0.341] · 30	0.535 [0.486, 0.585] · 30	**0.559** [0.502, 0.620] · 30	+ 1.5×10^{-8}	+ 3.2×10^{-7}	0.73
SC2-500	0.012 [0, 0.037] · 1	0.125 [0.110, 0.139] · 29	0.558 [0.481, 0.636] · 30	**0.725** [0.668, 0.789] · 30	+ 1.5×10^{-8}	+ 1.5×10^{-8}	+ 0.0035

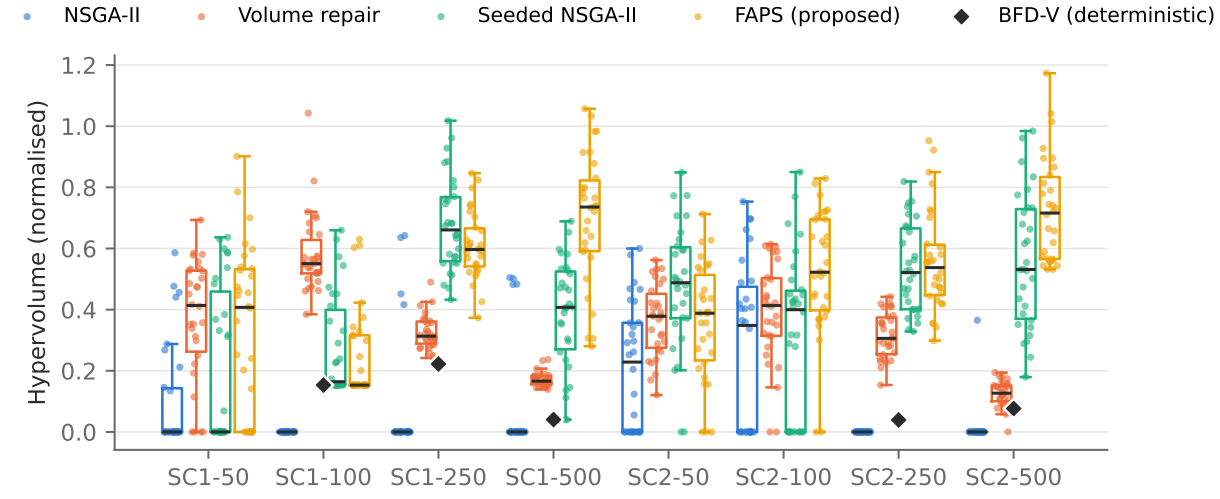


Fig. 3. Hypervolume per run (30 seeds). Diamonds mark BFD-V where it is feasible.

volume, matching SC1 and SC2. This gives 88 to 121 boxes per instance.

2. **Orientation.** The benchmark's orientation flags were kept, so a dimension may be vertical only if the benchmark permits it.
3. **Mass and category.** These are synthetic. Each box type was given a category drawn with SC1's probabilities and a density uniform on [100, 300] kg/m³. Box masses are density times volume, times lognormal noise with σ = 0.2.
4. **CoM target.** The container centre.

The extended-set problems were extracted from the archive and verified against it by checksum.

Cargo heterogeneity determines whether a single prior

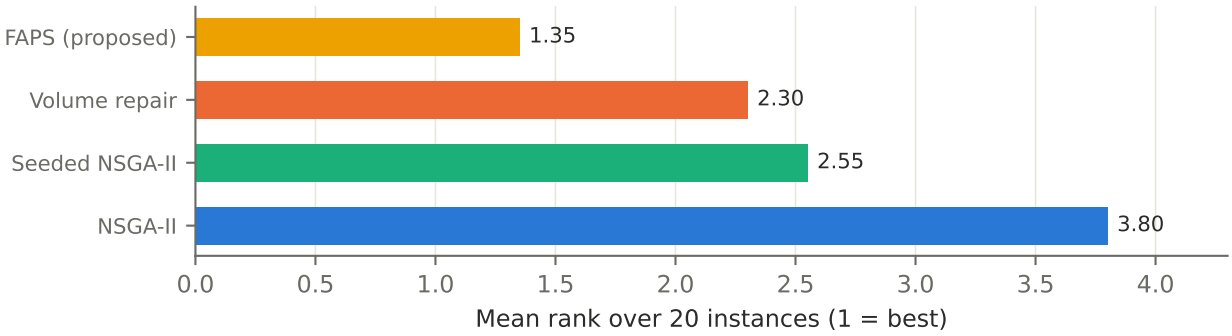


Fig. 4. Mean rank over the 20 instances (1 = best).

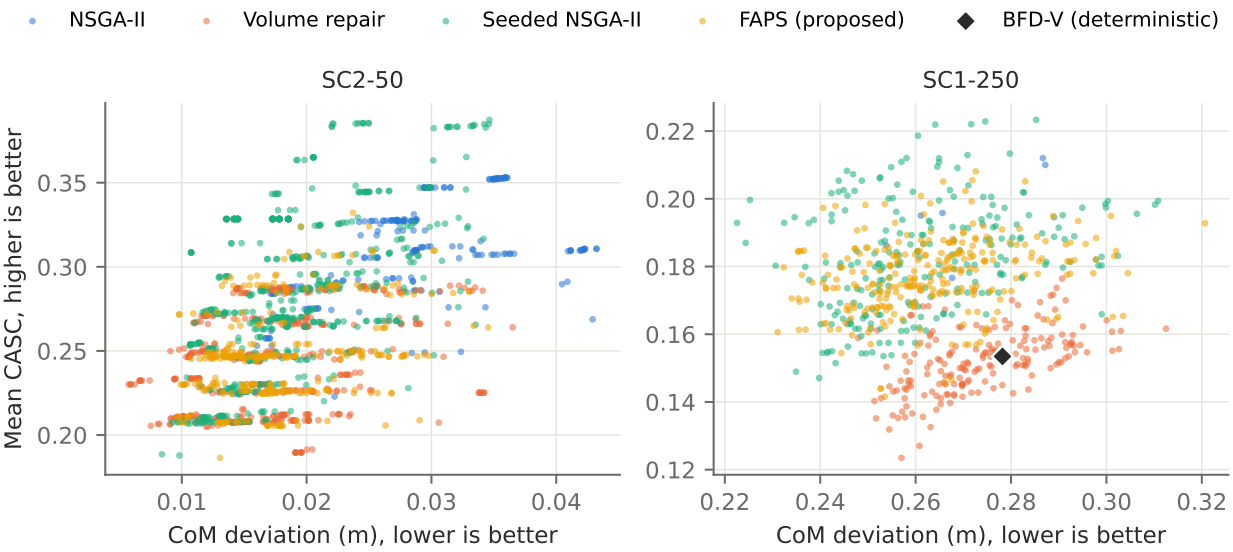


Fig. 5. Pooled run fronts on SC2-50 (left) and SC1-250 (right).

TABLE II
RANKING OVER ALL 20 INSTANCES (8 SYNTHETIC AND 12 BISCHOFF–RATCLIFF), RANKED PER INSTANCE BY MEAN HYPERVOLUME. FRIEDMAN $\chi^2 = 37.0$, $p = 4.5 \times 10^{-8}$. WINS COUNT THE INSTANCES ON WHICH FAPS'S MEAN HV EXCEEDS THE METHOD'S. THE P-VALUES ARE WILCOXON SIGNED-RANK TESTS OVER THE 20 INSTANCES.

Method	Mean rank	FAPS wins	p
FAPS	**1.35**	—	—
Volume repair	2.30	16 / 20	0.0017
Seeded NSGA-II	2.55	17 / 20	8.2×10^{-5}
Plain NSGA-II	3.80	20 / 20	1.9×10^{-6}

helps. With 3 box types, plain NSGA-II is usually feasible and the volume repair lowers front quality; on BR1-1's shared feasible seeds the repair reaches higher CoM deviation (0.141 against 0.072 m for FAPS) and lower accessibility. From 20 box types upward, plain NSGA-II never finds a valid packing and no constructive solution is feasible, so seeding yields only 0–5 feasible seeds and the repair becomes necessary.

FAPS across these regimes. When a seed is feasible (BR1-2, BR4-2), FAPS behaves like seeding with more seeds. On BR1-1 no seed is feasible but plain search sometimes is; the repair activates and FAPS reaches HV 0.538, against 0.273 for NSGA-II and 0.125 for the repair. On BR7–BR10, where neither seeds nor plain search are feasible, the repair activates and FAPS matches or exceeds it.

FAPS has the best mean on 11 of 12 instances, and is never significantly worse than any method. The exception, BR7-2, goes to the repair (0.329 against 0.293, Holm p = 1.0). With ten seeds and twelve instances in the Holm family, many large differences do not reach significance individually. The instance-level analysis in Section VI-B pools them.

E. A second optimiser: SMS-EMOA

To test whether the findings depend on NSGA-II, we re-ran plain search, the volume repair, single seeding and FAPS with SMS-EMOA on eight instances spanning both sets, with 10 seeds each.

Under SMS-EMOA the pattern matches NSGA-II. Plain search is rarely feasible, the repair wins on SC1-50 and SC1-100, seeding wins on SC1-250 and SC2-250 with FAPS close behind, and FAPS is clearly best on BR1-1 and BR4-2. FAPS

TABLE III

SYNTHETIC INSTANCES, RESTRICTED TO SEEDS ON WHICH BOTH METHODS ARE FEASIBLE. EACH CELL GIVES THE MEANS FOR THE TWO METHODS IN THE ORDER NAMED. THE BEST VALUES ARE THE PER-RUN MINIMUM COM DEVIATION (M) AND MAXIMUM MEAN CASC, AVERAGED OVER THESE SEEDS. P IS FROM A SEED-PAIRED WILCOXON TEST.

Instance	Comparison	Seeds	HV (p)	Best CoM dev. (p)	Best CASC (p)
SC1-50	repair vs NSGA-II	8	0.389 / 0.349 (0.64)	**0.019** / 0.032 (0.008)	0.257 / **0.304** (0.016)
SC2-50	repair vs NSGA-II	18	0.338 / 0.350 (0.80)	**0.014** / 0.027 (8×10^{-6})	0.230 / **0.286** (2.5×10^{-4})
SC2-100	repair vs NSGA-II	14	0.459 / 0.515 (0.30)	**0.056** / 0.071 (6×10^{-4})	0.212 / **0.248** (6×10^{-4})
SC1-250	repair vs seeded	30	0.327 / **0.679** (2×10^{-9})	0.260 / **0.243** (2×10^{-9})	0.143 / **0.170** (3×10^{-8})
SC2-500	repair vs seeded	29	0.147 / **0.652** (4×10^{-9})	0.651 / **0.584** (4×10^{-9})	0.091 / **0.128** (4×10^{-9})
SC1-50	FAPS vs repair	19	0.497 / 0.407 (0.13)	0.024 / **0.020** (0.026)	**0.291** / 0.257 (1×10^{-4})
SC2-100	FAPS vs repair	26	**0.559** / 0.412 (0.001)	0.059 / 0.059 (0.57)	**0.230** / 0.207 (0.007)
SC1-500	FAPS vs repair	30	**0.700** / 0.171 (2×10^{-9})	**0.608** / 0.688 (2×10^{-9})	**0.137** / 0.104 (2×10^{-9})
SC1-500	FAPS vs seeded	30	**0.700** / 0.388 (3×10^{-8})	**0.608** / 0.659 (5×10^{-8})	**0.137** / 0.126 (0.002)
SC2-500	FAPS vs seeded	30	**0.725** / 0.558 (5×10^{-4})	**0.568** / 0.584 (0.003)	**0.139** / 0.129 (0.015)
SC1-100	FAPS vs repair	30	0.248 / **0.581** (2×10^{-8})	0.063 / **0.057** (3×10^{-5})	0.173 / **0.189** (2×10^{-7})

TABLE IV

BISCHOFF–RATCLIFF INSTANCES, 10 SEEDS. EACH CELL GIVES THE MEAN HV WITH ITS BOOTSTRAP 95% INTERVAL AND THE NUMBER OF FEASIBLE SEEDS OUT OF 10. THE BEST MEAN PER ROW IS IN BOLD. FAPS'S LAST THREE COLUMNS GIVE HOLM-ADJUSTED P-VALUES (12 INSTANCES), MARKED + WHERE FAPS IS BETTER. BFD-V AND ALL FOUR PORTFOLIO CONSTRUCTIONS ARE INFEASIBLE ON EVERY INSTANCE EXCEPT BR1-2 AND BR4-2.

Instance	Types · N	NSGA-II	Volume repair	Seeded NSGA-II	FAPS	vs NSGA-II	vs repair	vs seeded
BR1-1	3 · 90	0.273 · 7	0.125 · 10	0.201 · 7	**0.538** [0.433, 0.686] · 10	0.094	+ 0.023	+ 0.023
BR1-2	3 · 111	0.774 · 10	0.723 · 10	0.790 · 10	**0.905** [0.827, 0.995] · 10	0.28	+ 0.023	0.33
BR4-1	10 · 88	0.352 · 5	0.409 · 10	0.365 · 10	**0.552** [0.436, 0.667] · 10	0.21	0.58	0.33
BR4-2	10 · 103	0.050 · 2	0.258 · 10	0.402 · 10	**0.581** [0.420, 0.751] · 10	+ 0.023	+ 0.023	0.33
BR7-1	20 · 94	0 · 0	0.341 · 7	0.142 · 2	**0.459** [0.212, 0.711] · 7	0.094	1.0	0.33
BR7-2	20 · 111	0 · 0	**0.329** · 8	0.094 · 3	0.293 [0.198, 0.384] · 10	+ 0.023	1.0	0.30
BR8-1	30 · 119	0 · 0	0.251 · 7	0.011 · 1	**0.361** [0.158, 0.596] · 7	0.094	1.0	0.14
BR8-2	30 · 114	0 · 0	0.274 · 7	0.139 · 3	**0.295** [0.137, 0.478] · 9	+ 0.031	1.0	0.33
BR9-1	40 · 121	0 · 0	0.391 · 8	0 · 0	**0.474** [0.320, 0.642] · 10	+ 0.023	1.0	+ 0.023
BR9-2	40 · 101	0 · 0	0.117 · 9	0.054 · 3	**0.181** [0.063, 0.305] · 6	0.094	1.0	0.33
BR10-1	50 · 109	0 · 0	0.346 · 6	0.239 · 5	**0.552** [0.370, 0.732] · 10	+ 0.023	1.0	0.33
BR10-2	50 · 98	0 · 0	0.307 · 9	0 · 0	**0.354** [0.202, 0.523] · 8	0.055	1.0	0.078

TABLE V

SMS-EMOA, 10 SEEDS. EACH CELL GIVES THE MEAN HV AND THE NUMBER OF FEASIBLE SEEDS. THE BEST MEAN PER ROW IS IN BOLD. THE LAST COLUMN GIVES FAPS AGAINST THE BEST OTHER METHOD, AS A HOLM-ADJUSTED P-VALUE OVER EIGHT INSTANCES.

Instance	Plain	Volume repair	Seeded	FAPS	FAPS vs best other
SC1-50	0.112 · 4	**0.387** · 8	0.230 · 5	0.309 · 6	vs repair: 1.0
SC1-100	0 · 0	**0.686** · 10	0.230 · 10	0.174 · 10	vs repair: − **0.020**
SC1-250	0.070 · 1	0.315 · 10	**0.704** · 10	0.693 · 10	vs seeded: 1.0
SC2-50	0.218 · 7	0.418 · 10	**0.523** · 9	0.483 · 10	vs seeded: 1.0
SC2-250	0 · 0	0.314 · 10	**0.622** · 10	0.616 · 10	vs seeded: 1.0
BR1-1	0.336 · 7	0.188 · 10	0.256 · 7	**0.630** · 10	vs plain: 0.098
BR4-2	0.056 · 2	0.366 · 10	0.446 · 10	**0.622** · 10	vs seeded: 1.0
BR7-2	0 · 0	**0.375** · 8	0.116 · 3	0.356 · 10	vs repair: 1.0

is significantly worse than the best competitor only on SC1-100, the same instance as under NSGA-II. Its improvement over plain SMS-EMOA is Holm-significant on five instances. The conclusions hold under a hypervolume-based survival rule as well as NSGA-II's.

F. Ablation of the volume repair

Six one-component ablations of the volume repair were run on SC1-100 and SC2-100 with seeds 1–10. Each ablated run's front was re-scored with the full objective and compared with the full repair, paired by seed, with Holm correction across the six ablations. HV here uses a reference front pooled over the ablation study.

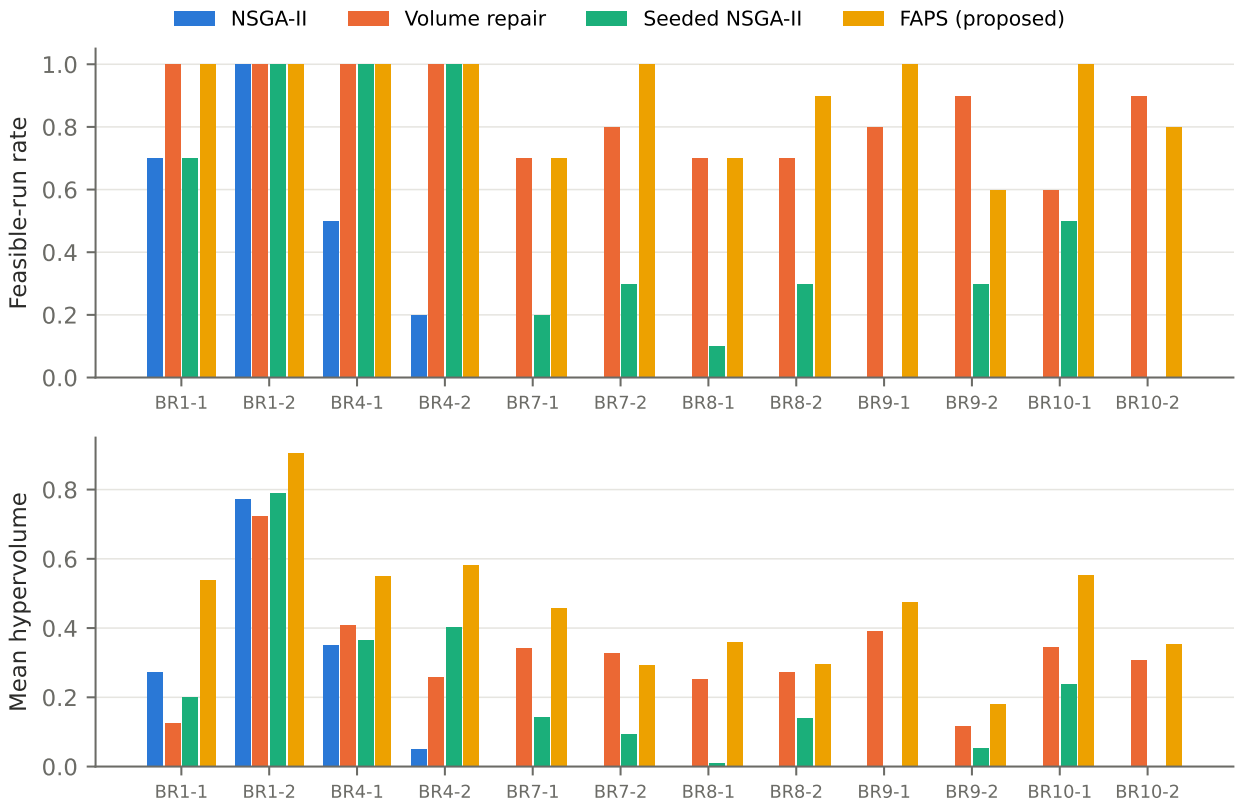


Fig. 6. Feasible-run rate (top) and mean hypervolume (bottom) on the Bischoff–Ratcliff instances.

TABLE VI
ABLATION RESULTS: HV MEAN ± SD, FEASIBLE SEEDS OUT OF 10, AND HOLM-ADJUSTED P AGAINST THE FULL REPAIR.

Variant	SC1-100 HV	Feas.	p	SC2-100 HV	Feas.	p
Full volume repair	0.758 ± 0.152	10	—	0.390 ± 0.193	9	—
Repair in random order	0	0	0.012	0.195 ± 0.259	4	0.64
No orientation search	0.028 ± 0.089	1	0.012	0.024 ± 0.051	2	0.023
Contact constraint	0.391 ± 0.136	10	0.012	0.064 ± 0.202	1	0.039
CASC without reach term	0.774 ± 0.162	10	0.85	0.397 ± 0.194	9	1.0
CASC without occlusion term	0.695 ± 0.119	10	0.11	0.395 ± 0.183	9	1.0
CASC without extraction term	0.661 ± 0.090	10	0.17	0.371 ± 0.164	9	1.0

A repair that re-decodes in random order is no better than no repair, so the gain comes from the decreasing-volume order itself; a second decoding attempt alone does nothing. Removing orientation search destroys feasibility on both instances. Removing any single CASC term from the search objective does not significantly change the full-objective front.

G. Contact-constrained packing

Under a contact constraint, each item away from the access face must rest at least half of its footprint on faces of items directly nearer the face. We re-ran plain NSGA-II, the volume repair, single seeding and the large-item repair with this constraint on the four synthetic instances with N $\leq$ 100 (10 seeds each). FAPS was not part of this study.

The contact constraint sharply reduces feasibility for every method. A constructive prior is still needed, and the repair and seeding still win on different instances, but the effect sizes from the unconstrained study should not be expected under contact physics.

H. Sensitivity to the objective definition

For the volume repair against plain NSGA-II, we re-decoded and re-scored every Pareto point under nine changed settings: each of L_arm, λ, C_grasp and κ halved and doubled, and the CoM target and dry mass moved to the container centre. BFD-V was re-run under each setting, and the statistics were recomputed. This analysis used the first study's seeds, 30 for

TABLE VII
CONTACT-CONSTRAINED DECODER: HV MEAN ± SD AND FEASIBLE SEEDS OUT OF 10.

Instance	NSGA-II	Volume repair	Seeded NSGA-II	Large-item repair	BFD-V
SC1-50	$0 \cdot 0$	$\mathbf{0.161 \pm 0.222} \cdot 5$	$0.116 \pm 0.264 \cdot 2$	$0.020 \pm 0.063 \cdot 1$	infeasible
SC1-100	$0 \cdot 0$	$\mathbf{0.478 \pm 0.144} \cdot 10$	$0.267 \pm 0.175 \cdot 10$	$0.414 \pm 0.315 \cdot 9$	0.185
SC2-50	$0.181 \pm 0.214 \cdot 5$	$0.206 \pm 0.206 \cdot 6$	$\mathbf{0.335 \pm 0.095} \cdot 10$	$0.141 \pm 0.156 \cdot 5$	0.158
SC2-100	$0.171 \pm 0.384 \cdot 3$	$0.074 \pm 0.234 \cdot 1$	$\mathbf{0.240 \pm 0.261} \cdot 5$	$0.158 \pm 0.264 \cdot 3$	infeasible

$N \leq 100$ and 10 for $N \geq 250$. With the original constants it reproduces that study exactly.

No change to the accessibility constants alters the direction of the comparison. Moving the CoM target reverses one instance, SC2-50. This analysis re-scores existing fronts without re-optimising, and it does not cover seeding or FAPS.

I. Computational cost

Decoder calls. The volume repair uses almost two decoder calls per evaluation, and seeded NSGA-II uses one, like plain NSGA-II. FAPS uses one per evaluation plus four constructions when a seed is feasible, and about 1.9 per evaluation once the repair activates at generation 10. The budget is matched in evaluations, so a time-matched comparison would favour single-decode methods. FAPS never uses more decoder calls than the volume repair, which it beats on 16 of 20 instances.

Wall-clock time. Later studies ran with different parallel loads, so wall-clock times are not compared. In the first study, the volume repair took 1.6–1.75 times NSGA-II's wall-clock time.

GPU scoring. On an NVIDIA RTX 4050 Laptop GPU, the corrected batched scorer evaluates 50, 100 and 250 packed SC1-100 layouts in 0.016, 0.041 and 0.108 s, against 0.101, 0.227 and 0.449 s on the CPU. These are medians of five repeats after warm-up, giving speedups of 6.4×, 5.5× and 4.1×. Packing remains on the CPU, and the whole-run speedup was not measured.

VII. DISCUSSION

Feasibility first. Hypervolume against a feasible reference front assigns 0 to a run without a valid packing. A method that is often infeasible therefore loses even when its feasible fronts are as good. Read alone, Table I suggests that the volume repair finds better trade-offs than plain NSGA-II. Table III shows that it finds valid packings more often, and that among valid packings it sits in a different region of the front with the same hypervolume. Evaluations of decoder-based packing MOEAs should report feasible-run rates and a feasibility-conditioned comparison alongside overall hypervolume.

Why FAPS works. The two standard priors fail in complementary regimes. Seeding needs a feasible constructive solution. The repair always restores feasibility, but it overrides the search on every failed evaluation and biases the trade-off. FAPS's portfolio raises the chance that some seed is feasible: the volume order fails on BR1-1 and SC2-100, yet the mass order succeeds on SC2-500, for example. Even infeasible seeds give the search diverse starting structure. The delayed repair supplies feasibility only when the search shows it cannot find any. The ablation shows why the fallback must be volume-ordered: a random-order repair gives no feasibility.

Where FAPS falls short. On SC1-100, under both optimisers, the repair's fronts are much better than those of any seeded method. There, a feasible seed exists, so FAPS never activates the repair, and the seeded population converges to a worse region than the repair reaches. Activating the repair whenever it helps would need a different trigger, for example one based on hypervolume progress. We leave that to future work.

Whether a prior is needed depends on the cargo. On near-homogeneous cargo (BR1) the decoder already finds valid packings, and a forced volume order constrains the search without helping it. On heterogeneous cargo (SC1, SC2, BR7–BR10) the decoder fails without help. FAPS covers both regimes by construction, so practitioners need not decide in advance.

Strong constructive baselines. BFD-V alone is feasible on seven of the twenty instances, including five synthetic ones where plain NSGA-II rarely or never is. A study comparing an MOEA only with random search would overstate the value of the search.

Equivalence checks. The scorer defect in Section V passed an equivalence check on fully packed layouts and showed up only on the partially packed layouts that dominate search, so accelerated scorers should be checked on the layouts the search actually visits.

VIII. THREATS TO VALIDITY AND LIMITATIONS

Internal validity.

- The Bischoff–Ratcliff, SMS-EMOA, ablation and contact studies use 10 seeds. On the Bischoff–Ratcliff set this limits per-instance significance under Holm correction across 12 instances. The instance-level Friedman analysis compensates only partly.
- FAPS's activation point (10% of generations) and its four orders were fixed before any FAPS run and not tuned. Different choices may do better or worse.
- Hyperparameters were not tuned for any method.
- The deterministic heuristics are single runs and are compared descriptively.
- The sensitivity analysis covers the repair-against-NSGA-II comparison only.
- The contact study does not include FAPS.

TABLE VIII
ROBUSTNESS OF THE REPAIR-AGAINST-NSGA-II COMPARISON (EIGHT INSTANCES).

Setting	Repair has higher mean HV	Holm-significant (α = 0.05)	Repair above BFD-V (of 5)
Original	8	7	5
L_arm $\times$ 0.5 / $\times$ 2	8 / 8	6 / 7	5 / 5
$\lambda \times$ 0.5 / $\times$ 2	8 / 8	7 / 7	5 / 5
C_grasp $\times$ 0.5 / $\times$ 2	8 / 8	6 / 7	5 / 5
$\kappa \times$ 0.5 / $\times$ 2	8 / 8	7 / 7	5 / 5
CoM target at container centre	7	6	5

- Wall-clock times were taken under different parallel loads, so we compare decoder calls instead.

Construct validity.

- CASC is an unvalidated surrogate for retrieval effort, and the CoM model is a rigid-body approximation.
- On the Bischoff–Ratcliff instances, masses and categories are synthetic, and the reach term saturates over most of the 5.87 m container, which flattens the accessibility objective there.
- Hypervolume values depend on the pooled method set, so they are compared only within a study.

External validity.

- The Bischoff–Ratcliff instances are subsets adapted to a complete-packing model, so the results are not comparable with published volume-utilisation results on that benchmark.
- Only two problems per class were used. Classes BR11–BR15 were not available in the archive we used.
- Launch-load restraint, static equilibrium [9] and soft goods are not modelled.
- Two optimisers were studied, both with the same decoder family.

Reproducibility. All code, instance files (including the Bischoff–Ratcliff source data, the extracted extended-set problems and the adaptation script), derived results and figures are in the repository. Objective values were reproduced exactly in a second environment. The raw evaluation archives (about 3 GB) are regenerated by the commands below.

IX. CONCLUSION

On heterogeneous cargo, decoder-based multi-objective search for CoM–accessibility packing rarely returns a valid packing on its own. The two standard ways of injecting a constructive packing order each fail somewhere. A Baldwinian volume repair restores feasibility but trades accessibility for compactness. A single seed improves the front at scale and does nothing when the constructive solution is infeasible. On near-homogeneous cargo, either can be worse than no prior.

Feasibility-adaptive portfolio seeding combines several constructive seeds with a repair that activates only if the search finds no valid packing early. It has the best mean rank over 20 instances (Friedman p = 4.5 $\times 10^{-8}$) and a higher mean hypervolume than every competitor on 16 to 20 of the 20 instances. On shared feasible seeds it recovers the accessibility the repair gives up, and it behaves the same under SMS-EMOA. It still loses to the repair on one instance under both optimisers, and its gains under contact physics were not tested.

Evaluations of packing optimisers should report feasibility separately from front quality and include strong constructive baselines. Natural next steps are a progress-based repair trigger, contact- and equilibrium-aware decoders, accessibility-aware construction orders, and validation of the surrogates against operational data.

REPRODUCIBILITY STATEMENT

The derived data behind every table and figure are in `docs/paper/data/`. The test suite (`pytest tests`, 52 tests) covers the scorer correction, every method variant including FAPS, the constructive seeds, the contact and ablation decoders, and the Bischoff–Ratcliff orientation flags.

Run the following from the repository root:

```
# Synthetic study (seeds 1-30): NSGA-II, repair, random, seeded, FAPS
python experiments/matched_budget_runner.py --datasets SC1-50 SC2-50 SC3-50 \
    SC1-100 SC2-100 SC3-100 --seeds 1 ... 30 --out experiments/full_matched
python experiments/matched_budget_runner.py --datasets SC1-250 SC2-250 SC3-250 \
    SC1-500 SC2-500 SC3-500 --seeds 1 ... 10 --out experiments/full_matched
python experiments/extended_runner.py --stage more_seeds --workers 10
python experiments/extended_runner.py --stage new_methods --workers 10
python experiments/heuristic_baselines.py --out experiments/full_matched

# Bischoff-Ratcliff instances, FAPS (all instances), SMS-EMOA, contact study
python experiments/br_instances.py
python experiments/extended_runner.py --stage br --workers 10
python experiments/extended_runner.py --stage faps --workers 10
python experiments/extended_runner.py --stage sms --workers 10
python experiments/extended_runner.py --stage contact --workers 10

# Analysis (repeat for experiments/br, experiments/sms, experiments/contact)
python experiments/matched_budget_analysis.py --dir experiments/full_matched
python experiments/conditional_analysis.py \
    --metrics experiments/full_matched/per_run_metrics.csv

# Sensitivity, ablation and figures (D = docs/paper/data)
python experiments/sensitivity_rescore.py --fronts $D/fronts_full_matched.npz \
    --out experiments/sensitivity
python experiments/ablation_study.py --datasets SC1-100 SC2-100 --seeds 1 ... 10
python experiments/ablation_analysis.py --ablation experiments/ablation \
    --fronts $D/fronts_full_matched.npz
python experiments/paper_figures.py --metrics $D/v9/synthetic/per_run_metrics.csv \
    --fronts $D/v9/synthetic/fronts_v9.npz --baselines experiments/full_matched \
    --br-metrics $D/v9/br/per_run_metrics.csv --ranks $D/v9/mean_hv_by_instance.csv

# Scoring-stage GPU timing (CUDA device required)
python experiments/benchmark_gpu_v2.py
```

APPENDIX

An earlier version of this work reported that NSGA-II significantly outperformed single-construction heuristics in hypervolume, and claimed a whole-run GPU speedup. Those results are withdrawn, for four reasons:

1. They were computed with the defective batched scorer (Section V).
2. Most reported solutions were infeasible.
3. The significance test compared 30 stochastic hypervolumes with a single fixed baseline value. That test asks

whether every run exceeds the value, which is a different question from whether the methods differ.

4. The GPU benchmark timed packing and scoring together, including start-up, and its equivalence check compared empty outputs.

References

[1] J. F. Gonçalves and M. G. C. Resende, "A biased random key genetic algorithm for 2D and 3D bin packing problems," *Int. J. Prod. Econ.*, vol. 145, no. 2, pp. 500–510, 2013.

[2] K. Deb, "An efficient constraint handling method for genetic algorithms," *Comput. Methods Appl. Mech. Eng.*, vol. 186, no. 2–4, pp. 311–338, 2000.

[3] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, 2002.

[4] S. Salcedo-Sanz, "A survey of repair methods used as constraint handling techniques in evolutionary algorithms," *Comput. Sci. Rev.*, 2009, doi: 10.1016/j.cosrev.2009.07.001.

[5] T. Friedrich and M. Wagner, "Seeding the initial population of multi-objective evolutionary algorithms: A computational study," *Appl. Soft Comput.*, vol. 33, pp. 223–230, 2015.

[6] E. E. Bischoff and M. S. W. Ratcliff, "Issues in the development of approaches to container loading," *Omega*, vol. 23, no. 4, pp. 377–390, 1995.

[7] A. Bortfeldt and G. Wäscher, "Constraints in container loading—A state-of-the-art review," *Eur. J. Oper. Res.*, vol. 229, no. 1, pp. 1–20, 2013.

[8] A. P. Davies and E. E. Bischoff, "Weight distribution considerations in container loading," *Eur. J. Oper. Res.*, vol. 114, no. 3, pp. 509–527, 1999.

[9] A. G. Ramos, J. F. Oliveira, J. F. Gonçalves, and M. P. Lopes, "A container loading algorithm with static mechanical equilibrium stability constraints," *Transp. Res. B, Methodol.*, vol. 91, pp. 565–581, 2016.

[10] S. Ceschia and A. Schaerf, "Local search for a multi-drop multi-container loading problem," *J. Heuristics*, vol. 19, no. 2, pp. 275–294, 2013.

[11] X. Zhao, J. A. Bennell, T. Bektaş, and K. Dowsland, "A comparative review of 3D container loading algorithms," *Int. Trans. Oper. Res.*, vol. 23, no. 1–2, pp. 287–320, 2016.

[12] S. Martello, D. Pisinger, and D. Vigo, "The three-dimensional bin packing problem," *Oper. Res.*, vol. 48, no. 2, pp. 256–267, 2000.

[13] T. G. Crainic, G. Perboli, and R. Tadei, "Extreme point-based heuristics for three-dimensional bin packing," *INFORMS J. Comput.*, vol. 20, no. 3, pp. 368–384, 2008.

[14] F. Parreño, R. Alvarez-Valdes, J. M. Tamarit, and J. F. Oliveira, "A maximal-space algorithm for the container loading problem," *INFORMS J. Comput.*, vol. 20, no. 3, pp. 412–422, 2008.

[15] G. Fasano, C. Lavopa, D. Negri, and M. C. Vola, "CAST: A successful project in support of the International Space Station logistics," in *Optimized Packings with Applications* (Springer Optimization and Its Applications). Cham, Switzerland: Springer, 2015, pp. 87–117.

[16] G. Fasano, *Solving Non-standard Packing Problems by Global Optimization and Heuristics* (SpringerBriefs in Optimization). Cham, Switzerland: Springer, 2014.

[17] K. Ho, "Space logistics modeling and optimization: Review of the state of the art," arXiv:2306.01107, 2023, doi: 10.2514/1.A35982.

[18] D. Whitley, V. S. Gordon, and K. Mathias, "Lamarckian evolution, the Baldwin effect and function optimization," in *Parallel Problem Solving from Nature—PPSN III* (LNCS 866). Berlin, Germany: Springer, 1994.

[19] J. Blank and K. Deb, "pymoo: Multi-objective optimization in Python," *IEEE Access*, vol. 8, 2020, doi: 10.1109/ACCESS.2020.2990567.

[20] E. Zitzler and L. Thiele, "Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach," *IEEE Trans. Evol. Comput.*, vol. 3, no. 4, pp. 257–271, 1999.

[21] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, and V. G. da Fonseca, "Performance assessment of multiobjective optimizers: An analysis and review," *IEEE Trans. Evol. Comput.*, vol. 7, no. 2, pp. 117–132, 2003.

[22] J. Derrac, S. García, D. Molina, and F. Herrera, "A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms," *Swarm Evol. Comput.*, vol. 1, no. 1, pp. 3–18, 2011.

[23] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[24] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychol. Bull.*, vol. 114, no. 3, pp. 494–509, 1993.

[25] S. O. Özcan, T. Dokeroğlu, A. Coşar, and A. Yazıcı, "A novel grouping genetic algorithm for the one-dimensional bin packing problem on GPU," in *Computer and Information Sciences (ISCIS 2016)*. Cham, Switzerland: Springer, 2016.